ServEase Viva Preparation Handbook

Steps 2 to 12

ServEase Viva Preparation Handbook (Steps 2 to 12)

Project: ServEase
Codebase analyzed from workspace folders `backend/` and `frontend/`.

2. TECH STACK ANALYSIS

2.1 Frontend Technologies

React (with functional components + hooks)

• What it does:
• Builds component-based UI and handles stateful views.
• Why used in your project:
• Multi-page app with role-based views: client dashboard, worker dashboard, worker settings, map view, auth pages.
• Implemented in `frontend/src/pages/*.jsx` and `frontend/src/components/*.jsx`.
• Alternatives:
• Vue, Angular, Svelte, Next.js.
• Why your choice is good/bad:
• Good: Fast development, clear component separation, rich ecosystem.
• Bad: Current app has some state duplication across pages and no global caching layer (like React Query), so repeated API calls happen.

React Router DOM

• What it does:
• Client-side routing and route guards.
• Why used here:
• Routes are defined in `frontend/src/App.jsx` and protected with `frontend/src/components/ProtectedRoute.jsx`.
• Alternatives:
• TanStack Router, Next.js App Router.
• Why choice is good/bad:
• Good: Simple role-based route control.
• Bad: Access control logic is split between component redirects and route guard checks, which can become harder to maintain as routes grow.

Axios

• What it does:
• HTTP client for REST API calls.
• Why used here:
• Centralized client in `frontend/src/utils/api.js` with interceptors for JWT attach and 401 handling.
• Alternatives:
• Fetch API + wrappers, ky, superagent.
• Why choice is good/bad:

• Good: Interceptors made token handling and global error behavior easy.
• Bad: Error parsing still repeated in some pages instead of one reusable helper.

Socket.IO Client

• What it does:
• Real-time bidirectional communication with backend socket server.
• Why used here:
• For hire request updates, worker availability changes, and location updates in:
• `frontend/src/pages/Dashboard.jsx`
• `frontend/src/pages/WorkerDashboard.jsx`
• `frontend/src/pages/WorkerSettings.jsx`
• `frontend/src/pages/MapView.jsx`
• Alternatives:
• Native WebSocket, Pusher, Firebase Realtime Database.
• Why choice is good/bad:
• Good: Automatic reconnection and event-based architecture.
• Bad: Multiple pages instantiate independent sockets; a centralized socket context would reduce duplication.

@react-google-maps/api

• What it does:
• Integrates Google Maps in React.
• Why used here:
• Live worker map and marker interactivity in `frontend/src/pages/MapView.jsx`.
• Alternatives:
• Mapbox GL, Leaflet, OpenLayers.
• Why choice is good/bad:
• Good: Familiar map UX and quick setup.
• Bad: API key handling currently has fallback default key in code; stronger secret management is recommended.

Tailwind CSS + PostCSS + Autoprefixer

• What it does:
• Utility-first styling and browser compatibility.
• Why used here:
• Rapid custom UI across pages; theme extension defined in `frontend/tailwind.config.js`.
• Alternatives:
• CSS modules, styled-components, Chakra UI, MUI.
• Why choice is good/bad:
• Good: Fast, consistent UI building.
• Bad: Large className strings can reduce readability.

Lucide React

• What it does:
• Icon library.
• Why used here:
• UI clarity in navigation, dashboards, status indicators.
• Alternatives:
• Heroicons, Font Awesome, Material Icons.
• Why choice is good/bad:
• Good: Lightweight and clean icon style.

• Bad: Minimal downside.

2.2 Backend Technologies

Node.js + Express

• What it does:
• Runtime and web framework for API and middleware pipeline.
• Why used here:
• REST APIs in `backend/routes/auth.js` and `backend/routes/workers.js`, server bootstrap in `backend/server.js`.
• Alternatives:
• NestJS, Fastify, Django, Spring Boot.
• Why choice is good/bad:
• Good: Fast iteration for startup/MVP style product.
• Bad: As codebase scales, architecture could benefit from stronger modular boundaries or service layers.

MongoDB + Mongoose

• What it does:
• NoSQL DB + schema modeling and querying.
• Why used here:
• Flexible models for users, workers, pending registrations, hires:
• `backend/models/User.js`
• `backend/models/Worker.js`
• `backend/models/PendingRegistration.js`
• `backend/models/Hire.js`
• Geospatial querying via `2dsphere` index in `Worker` model.
• Alternatives:
• PostgreSQL + Prisma, MySQL, DynamoDB.
• Why choice is good/bad:
• Good: Excellent fit for geospatial + flexible document structure.
• Bad: Some transactional workflows (hire state consistency) are harder than in relational systems.

JWT (jsonwebtoken)

• What it does:
• Stateless authentication token issuance and verification.
• Why used here:
• Login/register returns JWT; middleware validates token in `backend/middleware/auth.js`.
• Alternatives:
• Session + Redis, OAuth provider tokens, Paseto.
• Why choice is good/bad:
• Good: Scales well for stateless APIs.
• Bad: Token revocation is not fully implemented (logout is mostly client-side).

bcryptjs

• What it does:
• Password hashing.
• Why used here:
• Password security in register/reset flows in `backend/controllers/authController.js`.

- Alternatives:
- Argon2, scrypt.
- Why choice is good/bad:
- Good: Industry-standard and easy integration.
- Bad: Argon2 can be a stronger modern choice.

express-validator

- What it does:
- Request payload validation.
- Why used here:
- Input guards in route definitions before hitting controllers.
- Alternatives:
- Joi, Zod, Yup.
- Why choice is good/bad:
- Good: Route-level declarative validation.
- Bad: Validation logic spread across route files can get verbose.

express-rate-limit

- What it does:
- Throttles abusive request patterns.
- Why used here:
- General and auth-specific limiters in `backend/server.js`.
- Alternatives:
- Redis-based distributed limiter, NGINX limits.
- Why choice is good/bad:
- Good: Protects brute-force attempts quickly.
- Bad: In-memory limiter is less accurate in multi-instance deployments.

Nodemailer

- What it does:
- Sends OTP and reset emails.
- Why used here:
- Email utilities in `backend/utils/email.js`.
- Alternatives:
- SendGrid, SES, Mailgun.
- Why choice is good/bad:
- Good: Easy for development.
- Bad: Gmail SMTP is not ideal for high-scale production.

Socket.IO (server)

- What it does:
- Real-time events with rooms and auth middleware.
- Why used here:
- Live updates in `backend/sockets/index.js` for hires and map status/location.
- Alternatives:
- WebSocket ws library, MQTT, managed pub/sub.
- Why choice is good/bad:
- Good: Room-based event routing is clean for worker/user channels.
- Bad: Horizontal scaling requires sticky sessions + adapter (Redis adapter not yet configured).

2.3 Build/Dev Tools

Vite
- Fast frontend dev server/build.
- Used in `frontend/package.json`, `frontend/vite.config.js`.

Nodemon
- Backend hot-reload in dev.
- Used in `backend/package.json`.

Concurrently
- Starts frontend and backend together from root.
- Used in root `package.json` scripts.

ESLint
- Linting setup in `frontend/eslint.config.js`.

3. COMPLETE EXECUTION FLOW

3.1 Signup with OTP flow
1. User opens `Login` page and chooses signup in `frontend/src/pages/Login.jsx`.
2. Frontend calls `POST /api/auth/send-otp` via Axios client `frontend/src/utils/api.js`.
3. Route `backend/routes/auth.js` validates fields and forwards to `authController.sendOTP`.
4. `sendOTP` in `backend/controllers/authController.js`:
   - checks user duplication (`User.findOne`)
   - manages pending registration (`PendingRegistration`)
   - generates OTP and expiry
   - stores pending doc
   - sends email through `sendOTPEmail` (`backend/utils/email.js`)
5. User enters OTP on `frontend/src/pages/EmailVerification.jsx`, calls `POST /api/auth/verify-otp`.
6. Backend verifies OTP and returns short-lived registration token.
7. User sets password on `frontend/src/pages/SetPassword.jsx`, calls `POST /api/auth/register`.
8. Backend verifies token purpose, creates hashed password user, creates worker profile if role is worker, returns JWT + user.
9. Frontend stores token via AuthContext `login`, then navigates dashboard.

3.2 Login flow
1. User submits email/password in `frontend/src/pages/Login.jsx`.
2. `POST /api/auth/login`.
3. Backend validates credentials and verification status in `authController.login`.
4. JWT + user sent back.
5. AuthContext stores token/user in localStorage.
6. Protected routes become accessible.

3.3 App initialization with token verification
1. `AuthProvider` in `frontend/src/context/AuthContext.jsx` runs on app load.
2. If token exists, calls `GET /api/auth/verify-token`.
3. `protect` middleware validates JWT and loads user.
4. Success restores session; failure clears local storage and sets error.

3.4 Worker listing flow

1. Client page `frontend/src/pages/Dashboard.jsx` collects location/filter/sort.
2. Calls `GET /api/workers` with query params.
3. `backend/controllers/workerController.js -> getWorkers`:
   • builds text and skill search
   • applies rating/price/availability filters
   • if coordinates exist: `$geoNear` + score calculation + sort + limit
   • fallback query if no nearby workers
4. Response shown as cards in dashboard.

3.5 Hire request + real-time notifications

1. Client clicks hire in dashboard or map (`Dashboard.jsx` / `MapView.jsx`).
2. `POST /api/workers/hire` creates `Hire` record.
3. Backend emits `hireRequest` to `worker_<workerId>` room.
4. Worker `WorkerDashboard.jsx` listening on socket receives event and refreshes list.
5. Worker responds (`accept`/`reject`) using socket event `hireResponse`.
6. Backend updates `Hire` status and emits `hireUpdate` to `user_<userId>` room.
7. Client dashboard shows toast update.

3.6 Worker availability + location flow

1. Worker toggles availability in `WorkerSettings.jsx`.
2. API call `PUT /api/workers/availability` updates DB.
3. Backend broadcasts `worker-availability-changed`.
4. Map and dashboards subscribed through sockets update UI.
5. Worker can emit `update-location`; backend updates geo location and broadcasts `worker-location-updated`.

3.7 Rating flow

1. Client opens `UserHires.jsx` and submits rating/review for accepted hire.
2. API `POST /api/workers/rate`.
3. Backend validates ownership/status, stores review, marks hire completed.
4. Backend recalculates worker average rating from hire documents.

3.8 Password reset flow

1. `ForgotPassword.jsx` calls `POST /api/auth/forgot-password`.
2. Backend stores reset token + expiry and sends reset email.
3. User opens reset link page `ResetPassword.jsx`, submits new password.
4. Backend validates token expiry and updates hashed password.

4. CODE WALKTHROUGH (FILE-BY-FILE)

4.1 Backend Root

`backend/server.js`

• Purpose:

• App bootstrap, middleware registration, DB connect, routes, socket server, error handling, graceful shutdown.

• Important logic examiner may ask:

- CORS allow-list + localhost dynamic check.
- Two limiters: general + auth limiter.
- Auth limiter skip for `/verify-token` and `/logout`.
- Dynamic port retry in dev if port in use.
- Socket CORS config mirroring API CORS.

`backend/test-api.js`

- Purpose:
- Manual API test for OTP endpoint.
- Viva angle:
- Demonstrates quick endpoint smoke testing without Postman.

`backend/test-email.js`

- Purpose:
- Validate SMTP config and send test mail.
- Viva angle:
- Useful for infrastructure verification in dev.

4.2 Backend Routes

`backend/routes/auth.js`

- Purpose:
- Auth endpoint declarations + payload validation.
- Key functions wired:
- `sendOTP`, `verifyOTP`, `register`, `login`, `forgotPassword`, `resetPassword`, `verifyToken`, `logout`.
- Likely cross-question:
- Why validate at route layer before controller?

`backend/routes/workers.js`

- Purpose:
- Worker/hire endpoints with role checks.
- Key protections:
- `protect` for authentication.
- `authorize('worker')` for worker-only routes.
- `authorize('admin')` for seed route.

4.3 Backend Controllers

`backend/controllers/authController.js`

- Purpose:
- Business logic for full auth lifecycle.
- Key methods:
- `sendOTP`: pending registration + email send.
- `verifyOTP`: retry count enforcement + token issue.
- `register`: token validation + user creation + worker profile backfill.
- `login`: credential check + verified check + worker profile backfill.
- `forgotPassword` / `resetPassword`.
- `verifyToken` / `logout`.
- High-probability viva questions:

- Why pending table instead of storing OTP in user table?
- Why JWT includes `{ user: { id } }` payload shape?
- How retryCount limits OTP abuse?

`backend/controllers/workerController.js`

- Purpose:
- Worker discovery, hire operations, rating, availability updates.
- Key methods:
- `normalizeSearchTerm` alias mapping (plumber->plumbing).
- `getWorkers` geo + scoring + fallback.
- `hireWorker`, `getWorkerHires`, `getUserHires`.
- `rateWorker` with average rating recomputation.
- `updateWorkerAvailability` with socket broadcast.
- High-probability viva questions:
- Explain score formula trade-off.
- Why fallback when geo results are empty?

4.4 Backend Middleware

`backend/middleware/auth.js`

- Purpose:
- JWT verification and role authorization.
- Key logic:
- Extract Bearer token.
- Verify JWT and load user without password.
- Reject stale tokens if user deleted.

`backend/middleware/error.js`

- Purpose:
- Standardized centralized error mapping.
- Key handled classes:
- CastError, duplicate key, validation errors, JWT errors, express-validator custom arrays.

4.5 Backend Models

`backend/models/User.js`

- Purpose:
- User identity and auth fields.

`backend/models/PendingRegistration.js`

- Purpose:
- Temporary OTP signup state before account creation.
- Key detail:
- TTL index on `otpExpires` for auto-cleanup.

`backend/models/Worker.js`

- Purpose:
- Worker profile details, location, pricing, availability.
- Key detail:
- `location` GeoJSON + `2dsphere` index enables geo queries.

`backend/models/Hire.js`

- Purpose:
- Client-worker transaction record with lifecycle status and rating fields.

4.6 Backend Socket Layer

`backend/sockets/index.js`

- Purpose:
- Authenticated real-time channel handling.
- Key events:
- `hireResponse`
- `update-location`
- `worker-status-changed`
- Room strategy:
- `worker_<workerProfileId>` for worker-targeted events.
- `user_<userId>` for user-targeted updates.

4.7 Backend Utilities

`backend/utils/email.js`

- Purpose:
- OTP and reset-email template senders.

`backend/utils/response.js`

- Purpose:
- Unified API success/error response shapes.

4.8 Frontend Core

`frontend/src/main.jsx`

- Purpose:
- App bootstrap and AuthProvider wrapping.

`frontend/src/App.jsx`

- Purpose:
- Route mapping and protected path definitions.

`frontend/src/context/AuthContext.jsx`

- Purpose:
- Session state, verify-on-load, login/logout handling.

`frontend/src/utils/api.js`

- Purpose:
- Central Axios instance with:
- auth header injection
- 401 auto-handling
- local dev backend port discovery retry

4.9 Frontend Shared Components

`frontend/src/components/Navbar.jsx`
- Purpose:
- Role-aware navigation and logout action.

`frontend/src/components/ProtectedRoute.jsx`
- Purpose:
- Blocks unauthenticated users and enforces optional role.

`frontend/src/components/Toast.jsx`
- Purpose:
- Temporary status notifications across pages.

4.10 Frontend Pages

`frontend/src/pages/Landing.jsx`
- Purpose:
- Marketing/entry page with platform value proposition.

`frontend/src/pages/Login.jsx`
- Purpose:
- Combined login + signup start (send OTP).

`frontend/src/pages/EmailVerification.jsx`
- Purpose:
- OTP entry + resend flow with cooldown.

`frontend/src/pages/SetPassword.jsx`
- Purpose:
- Final signup completion with verification token.

`frontend/src/pages/ForgotPassword.jsx`
- Purpose:
- Start password reset.

`frontend/src/pages/ResetPassword.jsx`
- Purpose:
- Complete reset using URL token.

`frontend/src/pages/Dashboard.jsx`
- Purpose:
- Client worker listing, filters, hire trigger, hireUpdate socket listener.

`frontend/src/pages/MapView.jsx`
- Purpose:
- Interactive Google Map worker markers + booking from map + live socket updates.

`frontend/src/pages/WorkerDashboard.jsx`
- Purpose:
- Worker-side hire request management and response actions.

`frontend/src/pages/WorkerSettings.jsx`

- Purpose:
- Worker availability toggle and location sharing/live tracking.

`frontend/src/pages/UserHires.jsx`
- Purpose:
- Client hire history and rating submission.

4.11 Styling/Tooling files

`frontend/src/index.css`
- Tailwind base setup and theme variables.

`frontend/tailwind.config.js`
- Brand color extension and content scanning.

`frontend/eslint.config.js`
- Lint rules for React code quality.

5. CORE CONCEPTS (FROM YOUR CODE)

Routing
- Implemented using React Router in `frontend/src/App.jsx`.
- Routes are role-aware with `requiredRole` in `ProtectedRoute`.

API handling
- Centralized Axios in `frontend/src/utils/api.js`.
- All pages call same client for consistency.

Middleware
- Backend middleware chain:
- global request limiter
- CORS
- JSON parser
- auth limiter on `/api/auth`
- route handlers
- error handler
- Auth middleware (`protect`, `authorize`) in `backend/middleware/auth.js`.

Authentication
- JWT-based stateless auth.
- OTP pre-registration flow with separate pending model.
- Verify token endpoint for session restoration.

State management
- `AuthContext` for auth state globally.
- Local component state in each page for page-specific data.

Error handling
- Backend centralized via `backend/middleware/error.js` + `sendError` util.

- Frontend handles known API response structures and fallback messages.

Database operations

- CRUD via Mongoose in controllers.
- Geospatial queries in worker listing.
- Aggregation and computed scoring in worker matching pipeline.

6. WHY & DESIGN DECISIONS

Why this folder structure?

- Separation by concern:
- backend: routes/controllers/models/middleware/socket/utils
- frontend: pages/components/context/utils
- Easy for viva explanation and team contribution.

Why OTP + PendingRegistration instead of direct register?

- Prevents creating full user accounts before email ownership is proven.
- Cleaner lifecycle with temporary data auto-expiring via TTL index.

Why Socket.IO + REST together?

- REST handles durable actions (create hire, update availability).
- Socket handles instant notifications and live sync.
- This hybrid design is practical and scalable by concern.

Why geospatial matching in backend, not frontend?

- Avoids transferring huge worker lists to browser.
- Keeps ranking logic trusted, centralized, and optimized.

Why role-based authorization middleware?

- Security rule enforced server-side.
- Frontend redirects improve UX but are not security boundaries.

Trade-offs in current implementation

- Good:
- Clean MVP architecture with real-time + auth + maps.
- Strong practical feature completeness for viva.
- Limitations:
- No automated tests.
- In-memory rate limiting not distributed.
- No background job queue for emails.
- Socket lifecycle can be centralized better in frontend.

7. VIVA QUESTIONS (AT LEAST 40)

Basic (1-15)

1. What problem does ServEase solve?
2. Why did you choose MERN-style architecture?
3. How does your signup flow differ from a normal signup form?

4. Why do you need `PendingRegistration` model?
5. What is JWT and where do you use it?
6. How do you protect private routes in frontend?
7. How do you protect APIs in backend?
8. What is the purpose of `authorize('worker')` middleware?
9. Why are passwords hashed?
10. What is rate limiting and where did you apply it?
11. What is the role of `AuthContext`?
12. Why do you verify token on app load?
13. How does forgot-password work in your app?
14. What is Socket.IO used for in this project?
15. Why did you use MongoDB instead of SQL?

Intermediate (16-30)

16. Explain the full OTP verification sequence from frontend to DB.
17. How does your worker search handle both text and skill matching?
18. Why do you normalize search terms like plumber->plumbing?
19. Explain your sorting options in worker listing.
20. What is your AI score formula and why those factors?
21. Why do you have fallback logic when no nearby workers are found?
22. How do you ensure worker-only actions remain restricted?
23. What happens when token expires during active session?
24. How do your Axios interceptors improve reliability?
25. Why does backend try next port when one is busy in dev?
26. How is map marker data transformed from backend response?
27. How is hire acceptance/rejection propagated to user instantly?
28. How do you prevent rating someone else's hire request?
29. Why is `2dsphere` index important in Worker model?
30. How do you avoid stale worker accounts without profiles?

Advanced (31-50)

31. How would you scale Socket.IO to multiple servers?
32. What race conditions can happen in hire acceptance and how to fix?
33. How would you implement token revocation for JWT logout?
34. How would you harden OTP brute-force attempts further?
35. What consistency issues can occur when emitting socket events after DB writes?
36. How would you redesign for eventual consistency with queues?
37. How would you optimize worker query for 1M workers?
38. How would you move from Gmail SMTP to production-grade email?
39. What are security risks of storing JWT in localStorage?
40. How would you migrate to HttpOnly cookie auth?
41. How would you audit and trace user actions for compliance?
42. How would you design idempotency for hire creation requests?
43. How would you test real-time flows automatically?
44. How would you handle geolocation spoofing by workers?
45. How would you prevent abusive map polling/refresh patterns?
46. How would you redesign schema for multi-skill weighted ranking?
47. What if MongoDB geo index build fails in production?
48. How would you version APIs while keeping frontend backward compatible?

49. Why not GraphQL for this project?
50. If forced to use SQL, how would schema and geo-search change?

8. PERFECT ANSWERS (SHORT, CONFIDENT, NATURAL)

1. ServEase solves real-time discovery and booking of nearby service workers with trust and speed.
2. I used MERN because React gives fast UI iteration and MongoDB fits flexible worker/location documents.
3. My signup is 3-step: send OTP, verify OTP, then set password. Account is created only after verification.
4. `PendingRegistration` stores temporary signup data and auto-expires stale OTP attempts.
5. JWT is my stateless auth token; I issue it on login/register and verify in backend middleware.
6. Frontend uses `ProtectedRoute` and role checks in route declarations.
7. Backend secures APIs with `protect` middleware and role-based `authorize` checks.
8. It ensures only worker role can access worker-specific actions like viewing hire requests.
9. Hashing protects users if DB is leaked; plaintext passwords are never stored.
10. I apply rate limiting globally and stricter limits on auth routes to reduce brute-force abuse.
11. `AuthContext` keeps user session, loading state, login/logout helpers in one place.
12. Token verify on load restores valid sessions and cleans invalid tokens safely.
13. Forgot-password issues a time-limited reset token and sends a secure reset link by email.
14. Socket.IO is used for real-time hire alerts, status updates, and live worker location/availability.
15. MongoDB gave quick schema flexibility and native support for geospatial indexing.
16. Signup sends OTP, backend saves pending doc, user verifies OTP, backend issues short token, user sets password, account is created.
17. Search checks name and skills with regex and alias normalization for common terms.
18. It improves UX by matching user vocabulary to stored skill labels.
19. Users can sort by best match, distance, rating, or price and choose sort direction.
20. Score balances quality, affordability, and proximity to prioritize practical worker matches.
21. Fallback avoids empty UX by returning broader results when strict geo range has no workers.
22. Worker-only routes are server-enforced via role middleware, not just frontend redirects.
23. Axios catches 401, clears local session, and redirects to login with expiry context.
24. Interceptors centralize token attach/error handling and reduce repeated page logic.
25. It avoids dev downtime when default port is occupied and keeps startup smooth.
26. MapView maps backend GeoJSON `[lng,lat]` into Google marker `{lat,lng}` format.
27. Worker emits `hireResponse`; server updates DB and emits `hireUpdate` to user room.
28. Rating endpoint checks hire ownership and status before accepting user rating.
29. Without `2dsphere`, geo queries are slow or invalid; with it, `$geoNear` is efficient.
30. Login/register include worker profile backfill to keep old data compatible.
31. Use Redis adapter + sticky sessions + shared pub/sub event bus for Socket.IO scale-out.
32. Two workers accepting simultaneously can be prevented using atomic update with status guard.
33. Add token blacklist/denylist in Redis and verify token jti per request.
34. Add OTP attempt throttling per IP+email and challenge (captcha) for repeated failures.
35. If emit happens before commit confirmation, clients may see phantom state; emit after successful write.
36. Use queue-backed events so DB write and outbound notifications are retriable and decoupled.
37. Add geo + compound indexes, precomputed ranking fields, pagination, and cache hot regions.
38. Move to SES/SendGrid with verified domains, retries, and bounce monitoring.
39. localStorage tokens are vulnerable to XSS; HttpOnly cookies reduce JS-access risk.
40. Issue secure HttpOnly cookie tokens with CSRF protection and same-site controls.

41. Add audit logs for auth actions, hires, rating updates, and admin operations.
42. Use idempotency keys for hire creation endpoint to avoid duplicate accidental submissions.
43. Use integration tests with socket clients and deterministic event assertions.
44. Validate location changes with speed thresholds and periodic anti-fraud heuristics.
45. Add API throttles, websocket event throttling, and client debounce/backoff.
46. Separate skill taxonomy and weighted relevance model for finer scoring.
47. Health checks should fail deployment if geo index missing; run startup index validation.
48. Add `/v1`, `/v2` routes and keep additive response changes first.
49. REST is simpler here because use cases are action-centric and already map cleanly to endpoints.
50. In SQL, I'd use normalized tables + PostGIS for geo indexing and distance queries.

9. DEBUGGING & EDGE CASES

What can break

1. SMTP misconfiguration causes OTP/reset failure.
2. JWT expiry leads to sudden redirects if user is mid-action.
3. Socket auth failure if token missing/invalid in handshake.
4. Geolocation denied by browser.
5. Google Maps key invalid or quota exceeded.
6. Duplicate hire requests from rapid click.
7. Port mismatch between frontend and backend in local dev.
8. Worker profile missing for older worker user records.
9. Rate-limiter blocks during repeated testing.
10. Race conditions in hire state updates.

Fixes and prevention

1. Use provider-based email service and robust monitoring.
2. Add proactive token refresh strategy or pre-expiry warnings.
3. Centralize socket connection and reconnect with fresh token.
4. Graceful location fallback already present; add clear UX prompt.
5. Store key in env only and add map-load observability.
6. Add frontend button disable and backend idempotency key.
7. Already partially handled in Axios port discovery; document standard ports.
8. Keep current backfill logic and add migration scripts.
9. Support test-mode bypass or per-env limiter configs.
10. Use atomic DB transitions (status pending -> accepted once).

Common mistakes in viva discussion

- Saying frontend route guards are enough security (they are not).
- Claiming logout invalidates JWT server-side (current code does not blacklist).
- Ignoring geo index role in performance.

10. SCALABILITY & IMPROVEMENT PLAN

Near-term improvements

1. Add automated tests (unit + integration + socket e2e).
2. Add structured logging with request IDs.

3. Add API pagination and cursor-based results for worker lists.
4. Introduce React Query for caching and retries.
5. Use zod/joi schema centralization for cleaner validation.

1K -> 100K users

1. Deploy backend in containers behind load balancer.
2. Move limiter/session metadata to Redis.
3. Add Redis cache for hot worker search regions.
4. Add background queue for email jobs.
5. Add CDN and static asset optimization for frontend.

100K -> 1M users

1. Shard or partition data by geography.
2. Add read replicas and geo-distributed DB strategy.
3. Use dedicated search/ranking service for workers.
4. Use Redis adapter for Socket.IO cluster-wide events.
5. Introduce event-driven architecture for hire lifecycle.
6. Observability stack: tracing, metrics, SLOs, alerting.

Performance optimizations specific to your code

1. Prevent repeated socket instantiation across pages (central socket provider).
2. Reduce duplicate worker fetches on mount/location updates.
3. Precompute ranking attributes where possible.
4. Add projection fields to reduce payload size in worker list queries.

11. TRICKY EXAMINER-STYLE QUESTIONS

1. What if MongoDB is up but geo index is missing?
2. Why is your logout endpoint mostly symbolic with JWT?
3. If a worker goes offline after receiving hire request, what state guarantees remain?
4. What if two clients hire the same worker at the same second?
5. Why did you put alias normalization in backend and not frontend?
6. What happens if socket event succeeds but DB update fails?
7. What happens if DB update succeeds but socket emit fails?
8. Why should API authorization never depend on frontend role checks?
9. Why not store OTP on User model directly?
10. How do you defend against replay of reset tokens?
11. How do you detect abuse of live location updates?
12. Why did you use both API update and socket broadcast for availability?
13. Why do you trust geolocation coordinates from client side?
14. If CORS is open to localhost wildcard in dev, what is the risk?
15. How do you ensure consistency between worker room IDs and worker profile IDs?
16. Why is fallback result behavior better than empty list for user retention?
17. How would you preserve backward compatibility if response shape changes?
18. How would you monitor dropped socket events in production?
19. How would you migrate from localStorage JWT to cookie auth with minimum breakage?
20. What are your top three production risks right now?

12. MOCK VIVA MODE (INTERACTIVE)

How we will run it:

1. I ask one question.
2. You answer in your own words.
3. I grade your answer on:
   - Correctness
   - Confidence
   - Clarity
4. I improve your answer into a viva-ready version.
5. Then we move to the next question.

First mock viva question for you:

Explain why your project uses both REST APIs and Socket.IO instead of only one communication method. Use your own modules as examples.

(After this PDF, answer this question in chat and I will evaluate your response strictly.)

End of handbook.